CS 4400: Artificial Intelligence

Uninformed search

Contents

1. Tree search
2. Graph search
3. Breadth-first search
4. Uniform-cost search
5. Depth-first search
6. Modified depth-first search
7. Bidirectional search

Review

- Rational agents
- PEAS for task environment
  - Performance measure/environment/actuators/sensors
- Task environment
  - Observable/deterministic/single-agent/episodic/static
- Agent architecture
  - Reflex-based/model-based/goal-based/utility-based/learning

Review

Problem-solving by searching

observable, discrete, known, deterministic task environment → searching

Problem formulation

initial state/action/transition/goal test/path cost

Examples

Road to Bucharest, vacuum-world, 8-puzzle, 8-queens

1/7. Tree search

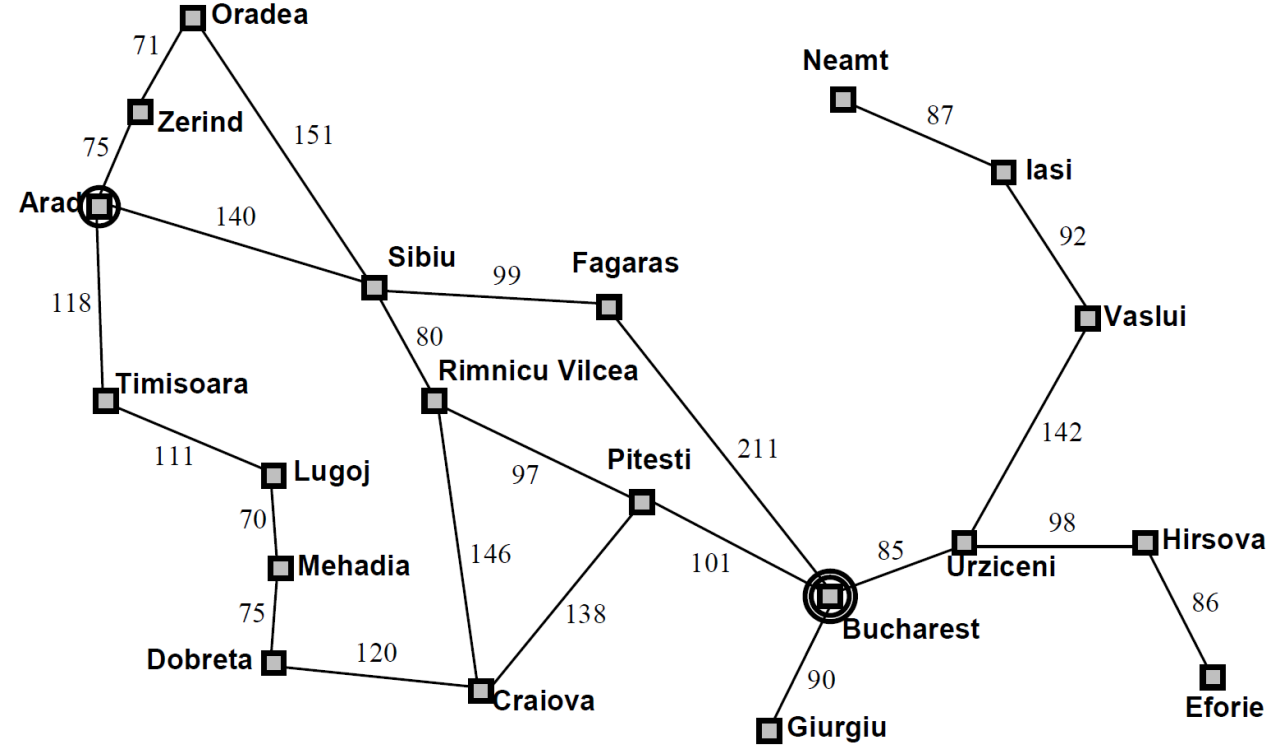
- Search tree
  - Root: initial state
  - Node: states
  - Branches: actions
- “Expanding a node”
  - Generate list of new states (=children) by considering all possible actions at the node
- Leaf node: node with no child
- Frontier: set of leaf nodes available for expansion (a.k.a. openlist)

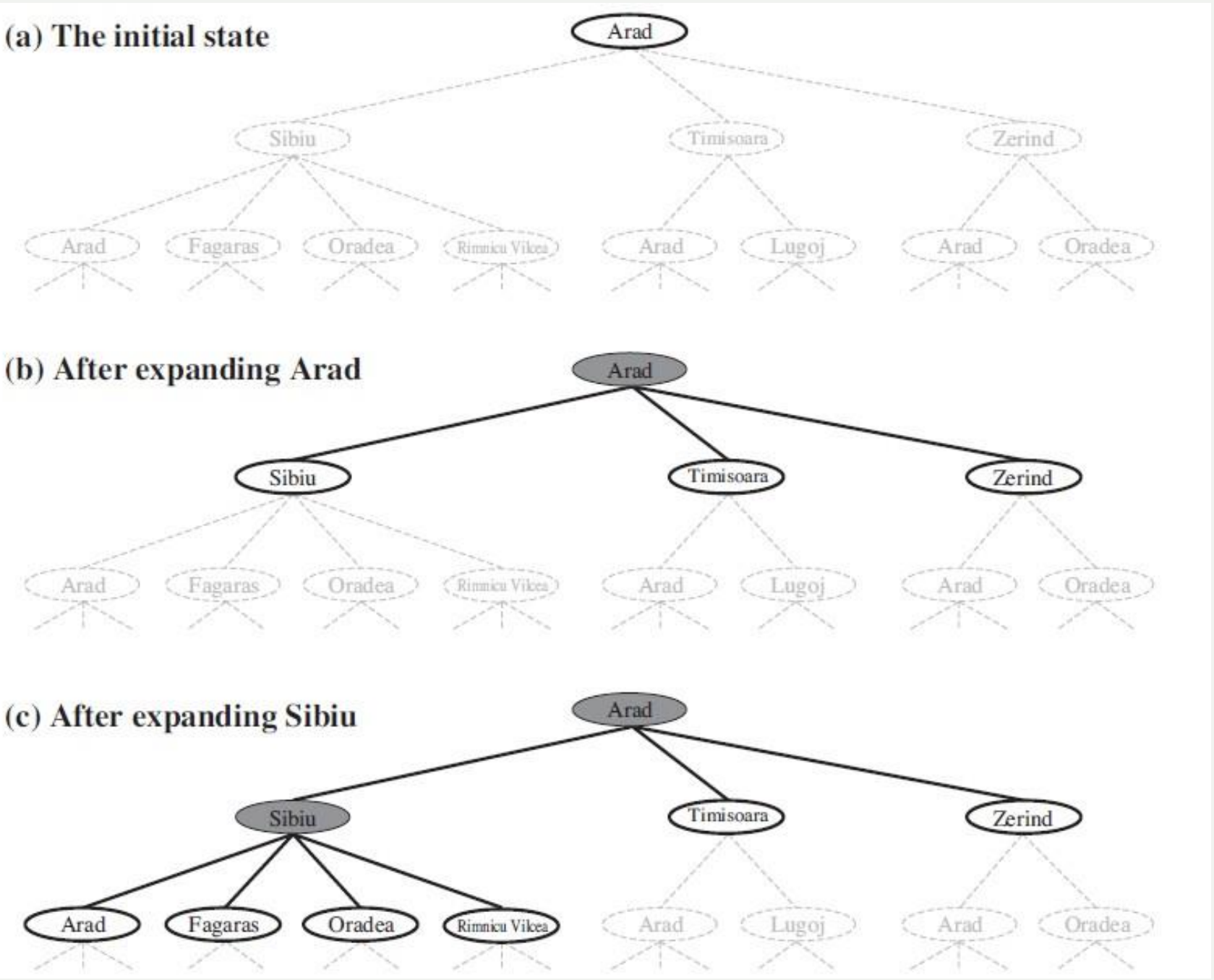
Tree search algorithms

- Basic idea:
  - Exploration of state space by generating successors of already-explored states (a.k.a.~expanding states).
  - Every states is evaluated: *is it a goal state*?

Example: Romania

From Arad to Bucharest




(a) The initial state
Arad
Sibiu
Timisoara
Zerind
Arad
Fagaras
Oradea
Rimnicu Vilcea
Arad
Lugoj
Arad
Oradea
(b) After expanding Arad
Arad
Sibiu
Timisoara
Zerind
Arad
Fagaras
Oradea
Rimnicu Vilcea
Arad
Lugoj
Arad
Oradea
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
Zerind
Arad
Fagaras
Oradea
Rimnicu Vilcea
Arad
Lugoj
Arad
Oradea

Tree search

function TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

Repeated states and redundant paths

- Search trees may have
  - Repeated states (= loopy path)
    - Arad-Sibiu-Arad
  - Redundant paths in general
    - Arad-Sibiu (140km)
    - Arad-Zerind-Oradea-Sibiu (297km)
- Redundant paths can sometimes be avoided by problem formulation
  - 8-queens
    - Init: empty board → final: n queens (n! paths to the final state)
    - Add a queen to the leftmost empty column (1 path)

Repeated states and redundant paths

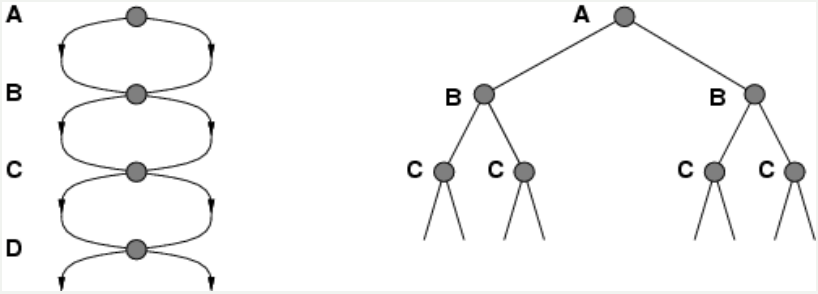
Example: rectangular grid

4 children for each node

After d-steps, say d=20

4^d = ~ trillion nodes

~$2d^2$ = 800 distinct states



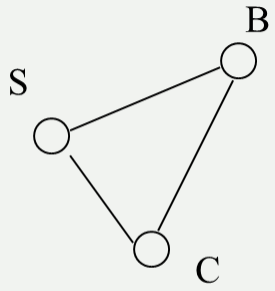
Tractable problem becomes intractable !

♠ Failure to detect repeated states can turn a solvable problem into an unsolvable one.

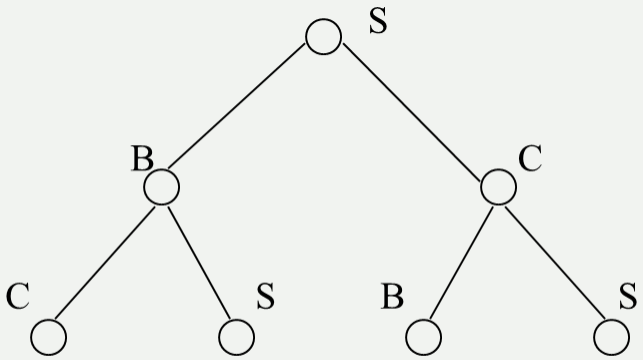
Cure: graph search

♦ Keep only the best path to each state.

Graph Search



State Space

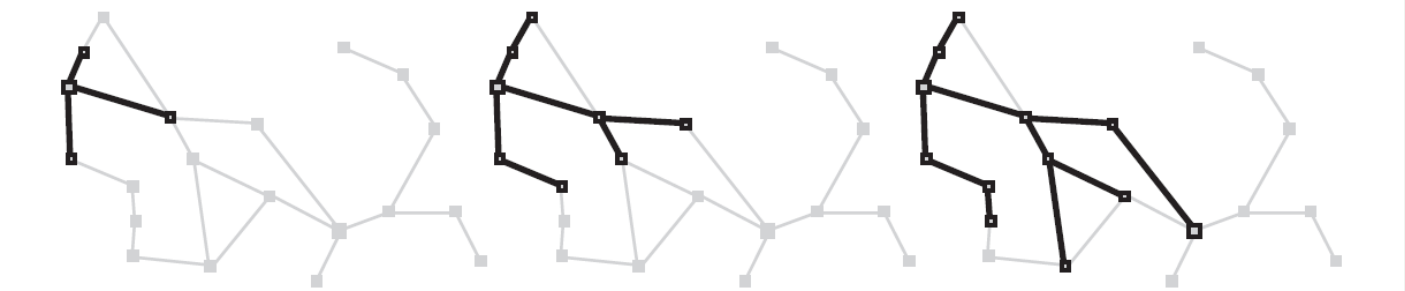


Example of a Search Tree

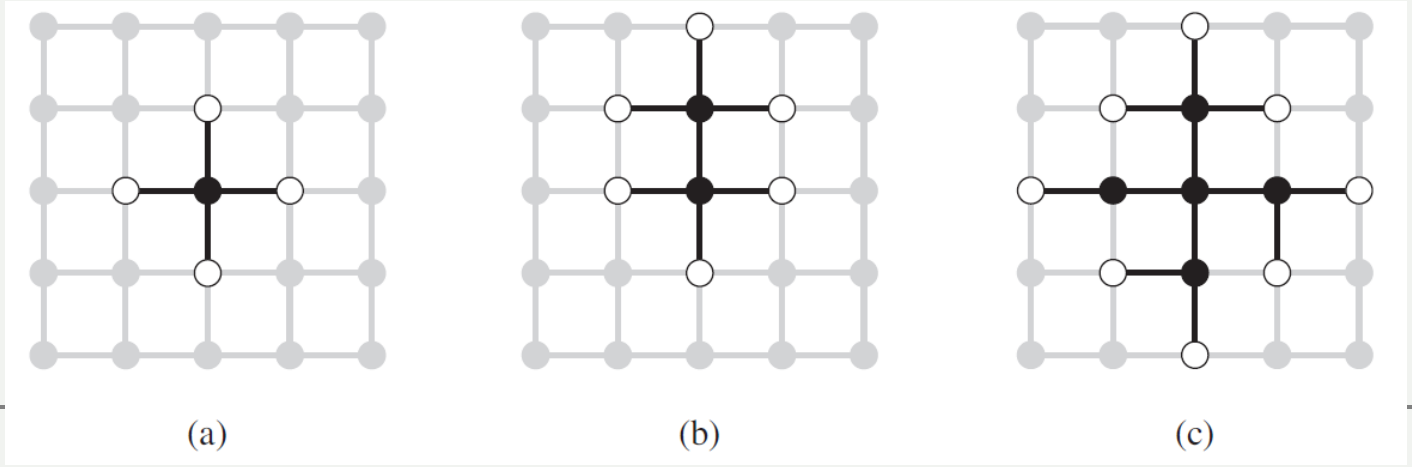
- Keep the explored set (a.k.a closed list)
- If a newly generated node has been explored before,
  do not add it to the frontier
- never generate a state generated before
  - must keep track of all possible states (uses a lot of memory)
  - e.g., 8-puzzle problem, we have 9! = 362,880 states
- Optimal but memory inefficient

2/7. Graph search

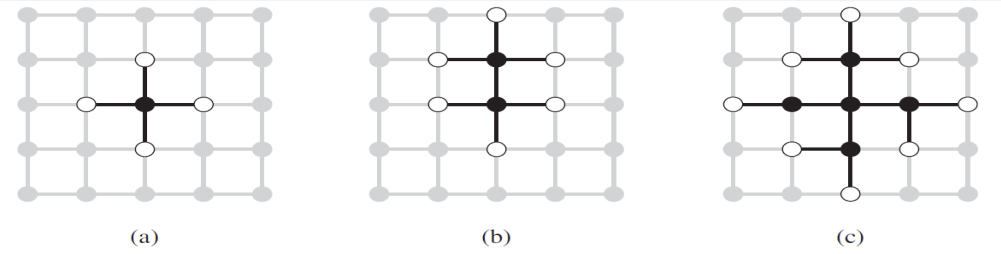
Search graph is a subgraph of state-space graph



Frontier separates explored and unexplored regions



Graph search



function GRAPH-SEARCH(*problem*) **returns** a solution, or failure

initialize the frontier using the initial state of *problem*

initialize the explored set to be empty

loop do

if the frontier is empty **then return** failure

choose a leaf node and remove it from the frontier

if the node contains a goal state **then return** the corresponding solution

add the node to the explored set

expand the chosen node, adding the resulting nodes to the frontier

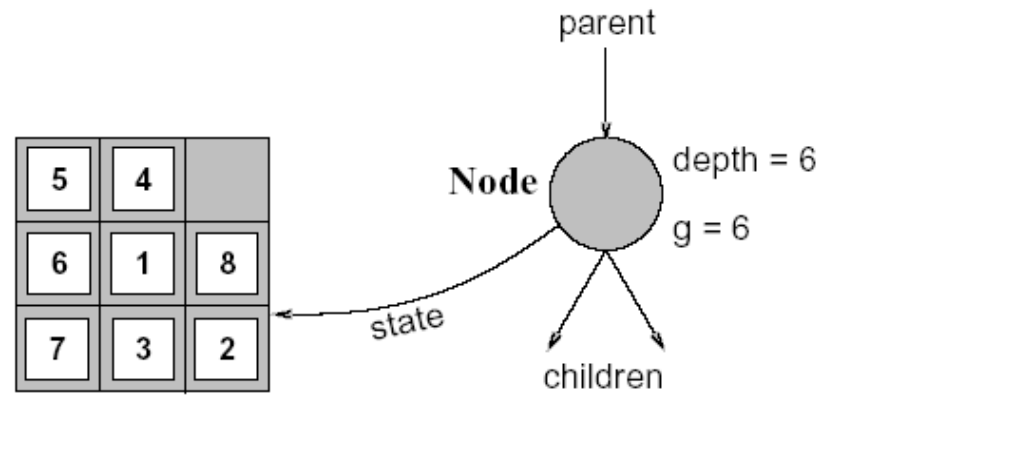
only if not in the frontier or explored set

Although graph search is more efficient, I will use tree search for demonstrations in class

Data structure

Node class

- State
- Parent node: which node generated this?
- Action: which action generated this?
- Path-cost [g(n)]: cost from init state to current state
- Depth



Data structure

Queue to store frontier, explored set, etc

Queue member functions

- EMPTY(q)
- POP(q): remove and return the first element
- INSERT(e,q): insert an element
- ISELEMENT(e,q): is the element inside the queue?

Queue types

- First-in first-out (FIFO): pops the oldest element
- Last-in first-out (LIFO): pops the newest element
- Priority queue: pops the element with highest priority

Measuring performance of strategy

Strategy(=algorithm): order of expanding nodes

Completeness: is the strategy guaranteed to find the solution (if there is one)?

Optimality: does it find the optimal (=lowest path-cost) solution?

Time and space complexities

$T(n)$ is $O(n^2)$ iff $T(n) < cn^2$ for some c, for all $n > n_0$

Time and space complexity

Time complexity = max # of nodes generated until goal

Tree search: # of nodes generated = # of nodes explored + # of nodes in frontier

Graph search: # of nodes generated ≤ # of states

(since no repeated state)

Space complexity = max # of nodes in memory until goal

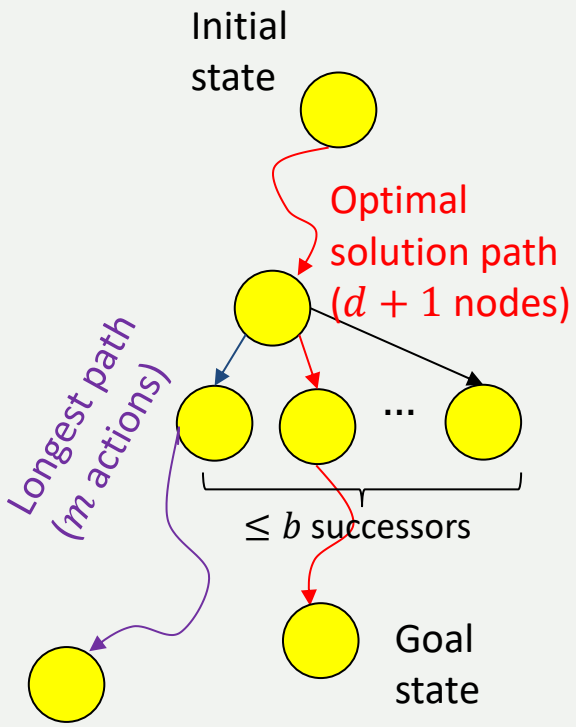
Tree search: # of nodes in frontier

Graph search: space complexity ≈ time complexity

(since all generated nodes are kept in memory)

Measuring performance of strategy

- Accordingly, complexity is measured in terms of
  - d: *depth* (number of actions in the optimal solution)
  - m: *maximum number of actions* in any path
  - b: *branching factor* (number of successors of a node).



Uninformed search

Uninformed search (a.k.a. systematic blind search)

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening depth-first search
- Bidirectional search

Informed search (a.k.a. heuristic search)

While searching you have no clue whether one non-goal state is better than any other. Your search is blind. You don't know if your current exploration is likely to be fruitful.

Complexity Recap

- We often want to characterize algorithms independent of their implementation.
- "*This algorithm took 1 hour and 43 seconds on my laptop".* Is not very useful, because tomorrow computers are faster.
- Better is:
  "This algorithm takes O(nlog(n)) time to run and O(n) to store". because this statement abstracts away from irrelevant details.
- Time(n) = O(f(n)) means:
  Time(n) < constant x f(n) for n>n0 for some n0
  Space(n) idem.
- n is some variable which characterizes the size of the problem,
  e.g. number of data-points, number of dimensions, branching-factor of search tree, etc.

3/7. Breadth-first search

Expand the shallowest unexpanded node

e.g., root, all children, all grandchildren,…

Queue: two ways to look at it

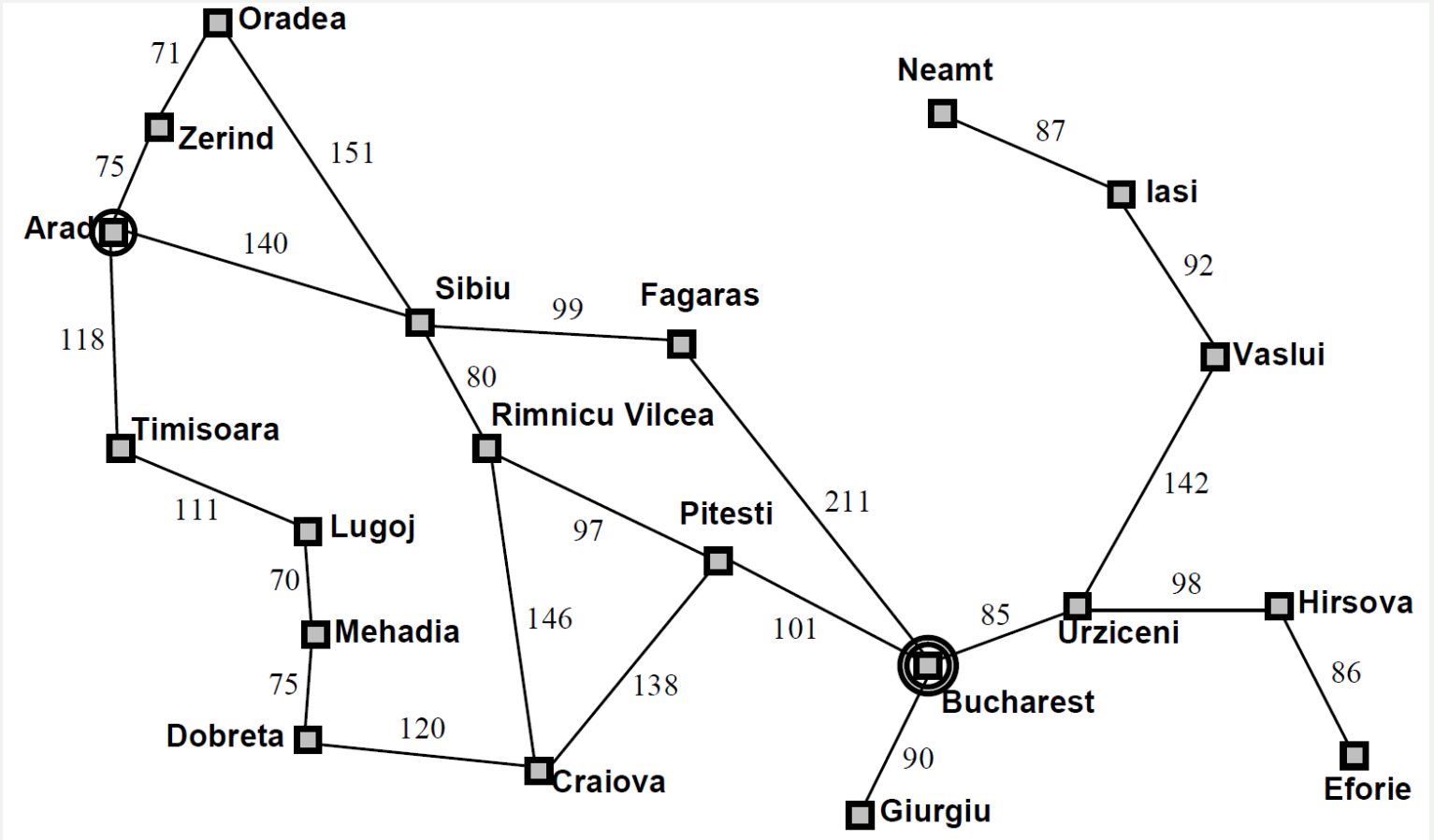
FIFO Queue

Priority queue: lowest depths first

Breadth-first search

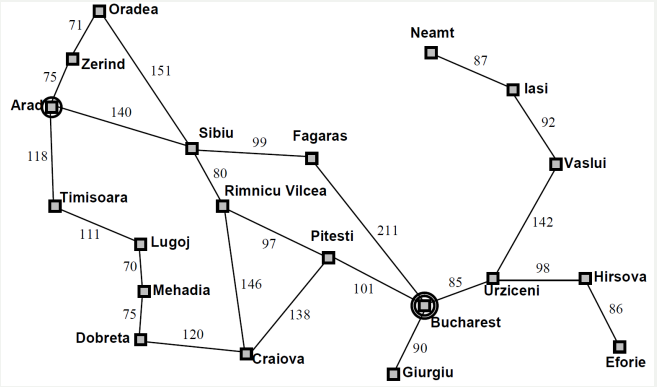
```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

Example: Romania



Planning trip from Arad





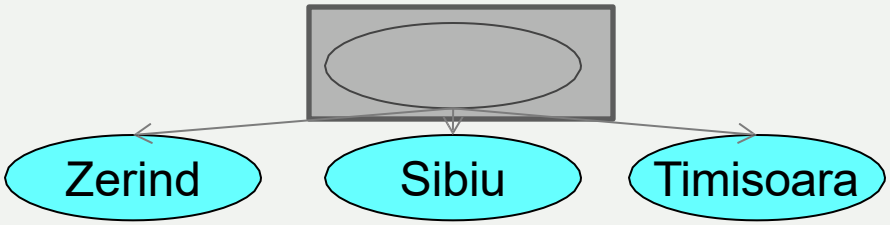
Node: `< State, Parent, Action, PathCost, Depth >`

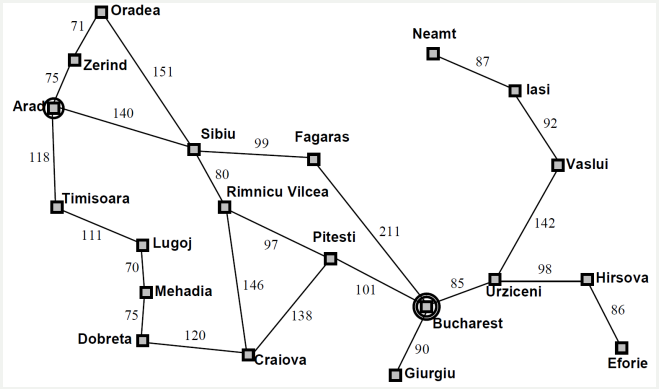
Assume `PathCost == Depth`

Init state is “Arad”: root.state = “Arad”

Queue: `{ node1 = < In(Arad), null, null, 0, 0 > }`

Planning trip from Arad





Expand: `node1 = < In(Arad), null, null, 0, 0 >`

Queue: {

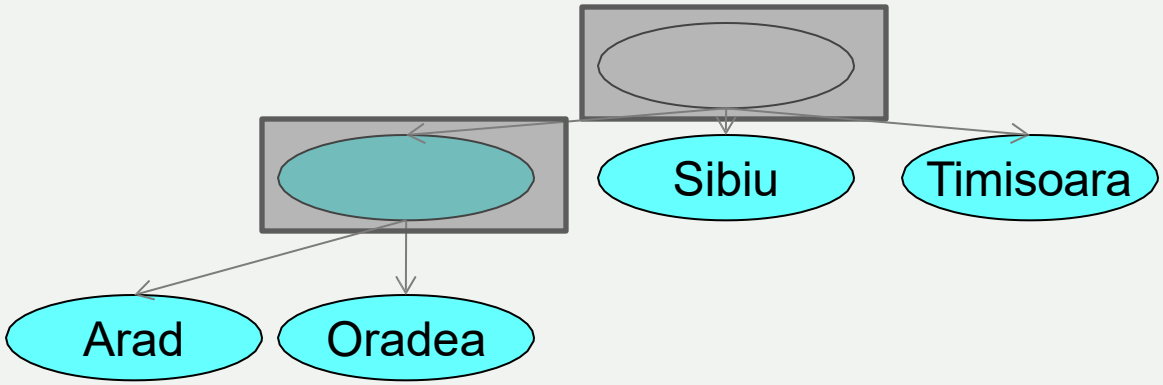
`node2 = < In(Zerind), node1, Go(Zerind), 75, 1 >,`

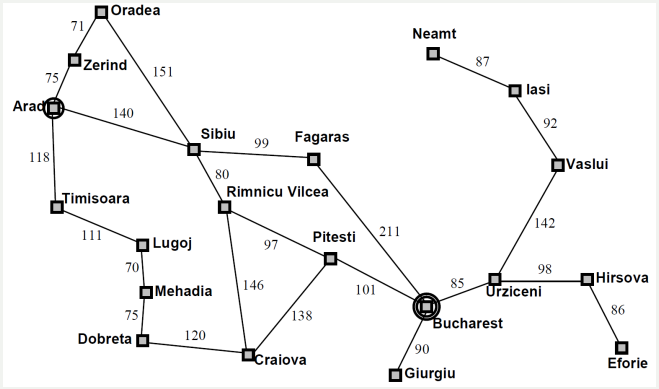
`node3 = < In(Sibiu), node1, Go(Sibiu), 140, 1 >,`

`node4 = < In(Timis…), node1, Go(Timis…), 118, 1 >`

}

Planning trip from Arad





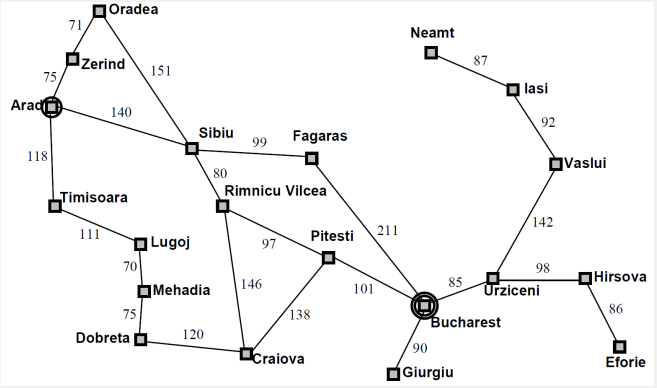
Expand: `node2 = < In(Zerind),node1,Go(Zerind),75, 1>`

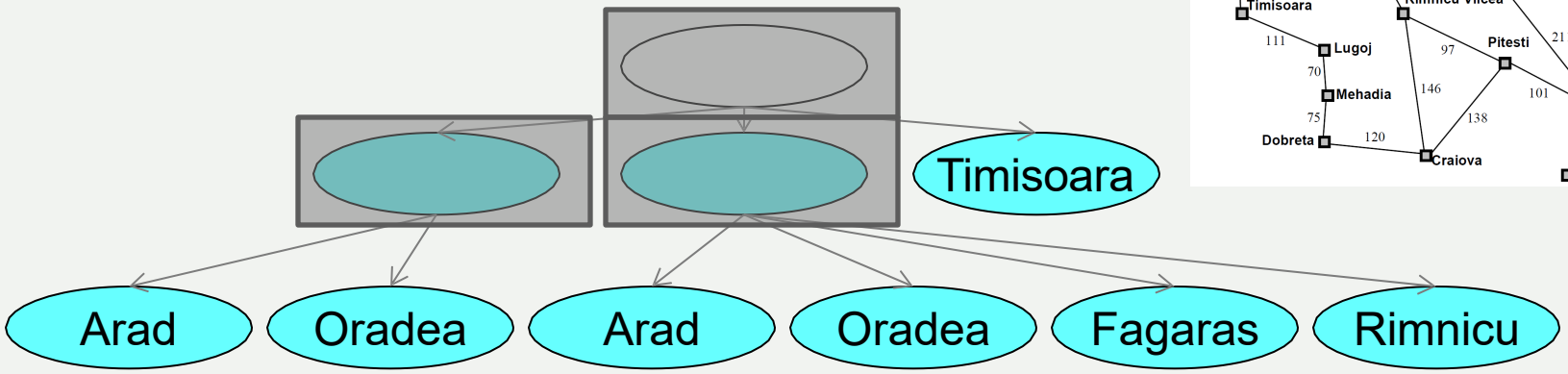
Queue: {

```
Node3 = < In(Sibiu), node1, Go(Sibiu), 140, 1 >,
Node4 = < In(Timis…), node1, Go(Timis…), 118, 1 >
Node5 = < In(Arad), node2, Go(Arad), 150, 2>
Node6 = < In(Oradea),node2, Go(Oradea), 146, 2>
```

}

Planning trip from Arad





Expand: `node3 = < In(Sibiu),node1,Go(Sibiu),140, 1 >`

Queue: {

```
Node4 = < In(Timis…), node1, Go(Timis…), 118, 1 >
Node5 = < In(Arad), node2, Go(Arad), 150, 2>
Node6 = < In(Oradea), node2, Go(Oradea), 146,2>
Node7 = < In(Arad), node3, Go(Arad), 280,2>
…
}
```

Properties of breadth-first search

Complete? Yes, if b is finite (it always reach goal)

Optimal?

Yes, if path cost is nondecreasing w.r.t. node depth

Time complexity:

nodes at depth n = b^n

$1+b+b^2+b^3+\ldots+b^d = O(b^d)$

《cf.》

$$S_n = a + ar + ar^2 + ar^3 + \ldots + ar^{n-1}$$
$$-) \; rS_n = \quad ar + ar^2 + ar^3 + \ldots + ar^{n-1} + ar^n$$
$$(1 - r)S_n = a \qquad\qquad\qquad\qquad\qquad - ar^n$$
$$S_n = \frac{a - ar^n}{1 - r} = \frac{ar^n - a}{r - 1} = \frac{a(r^n - 1)}{r - 1}$$

Space complexity:

of explored nodes at depth d = $O(b^{d-1})$

of frontier nodes at depth d = $O(b^d)$

Properties of breadth-first search

Tree search: $O(b^d)$

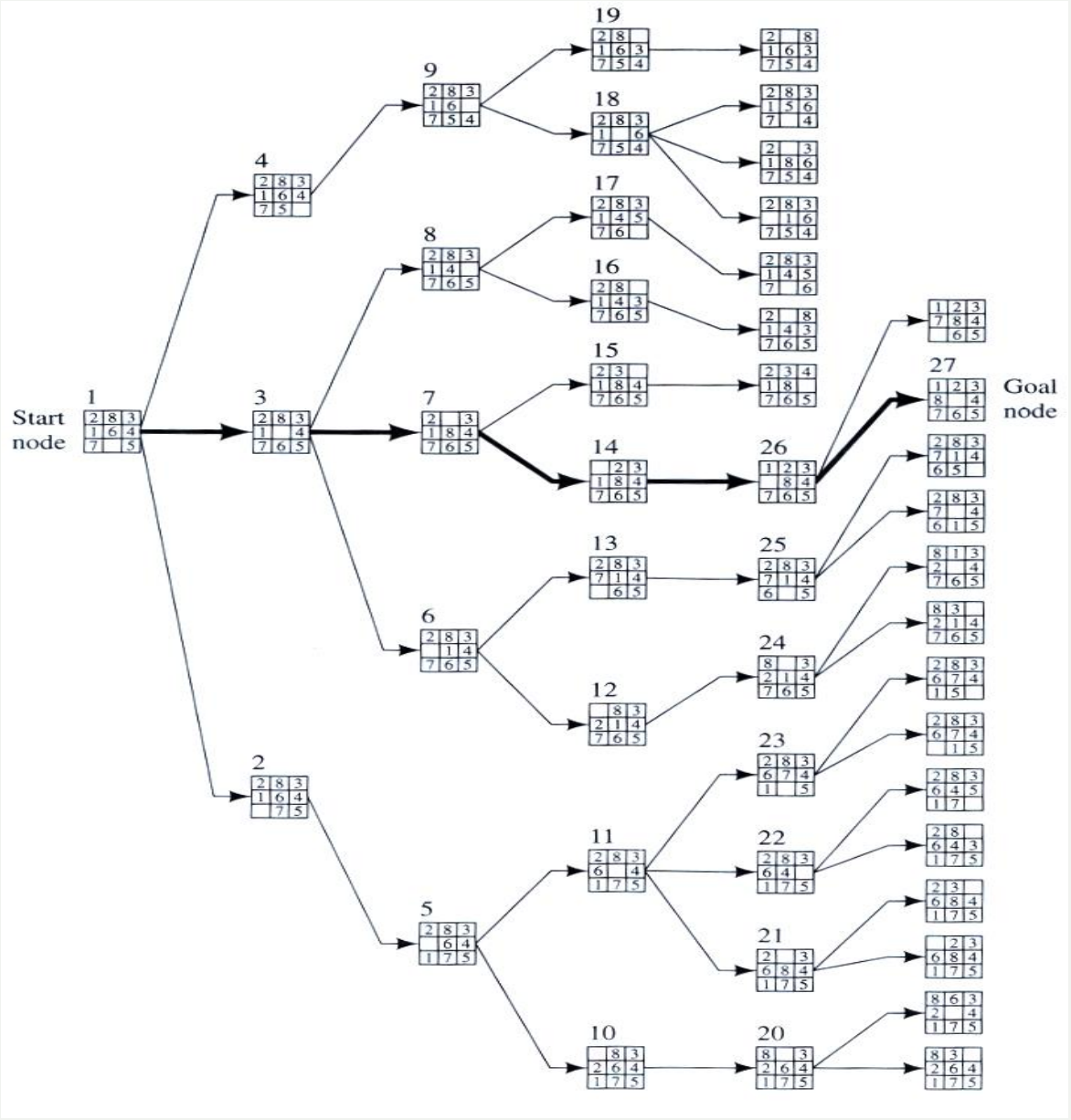
Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 KB
4	11,110	11 milliseconds	10.6 MB
6	10^6	1.1 seconds	1 GB
8	10^8	2 minutes	103 GB
10	10^{10}	3 hours	10 TB
12	10^{12}	13 days	1 PB
14	10^{14}	3.5 years	99 PB
16	10^{16}	350 years	10 EB

$b = 10$

♣ Only small search problems are solvable due to exponential time complexity.
♣ Memory is a bigger issue than time.

Graph search is preferred since its time and space are proportional to the size of the state space (often less than $O(b^d)$).

Example BFS


Start node
Goal node
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

4/7. Uniform-cost search

Breadth-first is only optimal if step costs is increasing with depth (e.g. constant).
Can we guarantee optimality for any step cost?

Uniform-cost Search: Expand node with smallest path cost g(n).

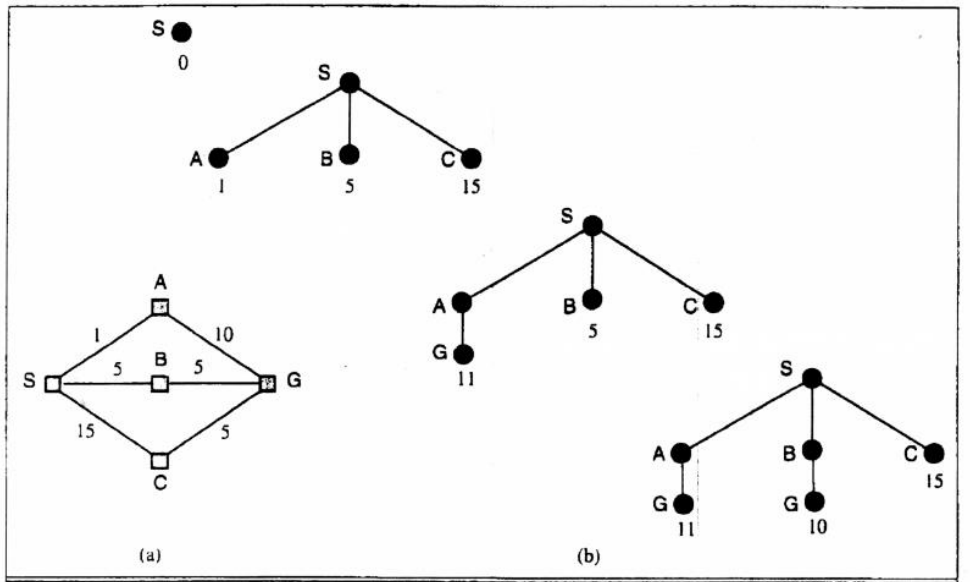


Figure 3.13 A route-finding problem. (a) The state space, showing the cost for each operator. (b) Progression of the search. Each node is labelled with $g(n)$. At the next step, the goal node with $g = 10$ will be selected.

Proof Completeness:

Given that every step will cost more than 0, and assuming a finite branching factor, there is a finite number of expansions required before the total path cost is equal to the path cost of the goal state. Hence, we will reach it.

Proof of optimality given completeness:

Assume UCS is not optimal.
Then there must be an (optimal) goal state with path cost smaller than the found (suboptimal) goal state (invoking completeness).
However, this is impossible because UCS would have expanded that node first by definition.
Contradiction.

Uniform-cost search

Implementation: queue ordered by path cost,
Equivalent to breadth-first if all step costs all equal.

Complete?

Time?

Space?

Optimal?

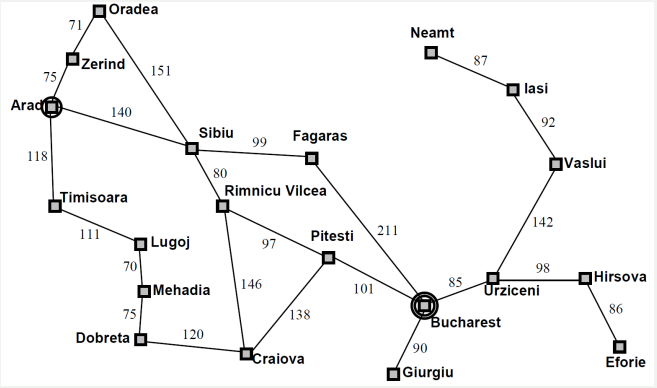
Uniform-cost search

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure

  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
      if EMPTY?(frontier) then return failure
      node ← POP(frontier)   /* chooses the lowest-cost node in frontier */
      if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
      add node.STATE to explored
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          if child.STATE is not in explored or frontier then
              frontier ← INSERT(child, frontier)
          else if child.STATE is in frontier with higher PATH-COST then
              replace that frontier node with child
```

If the child is not in any of these 'explored' or 'frontier'…

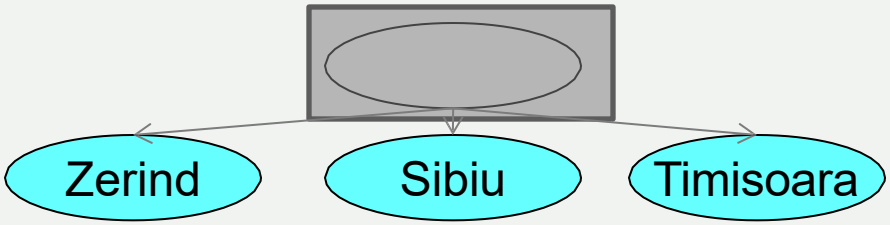
Planning trip from Arad

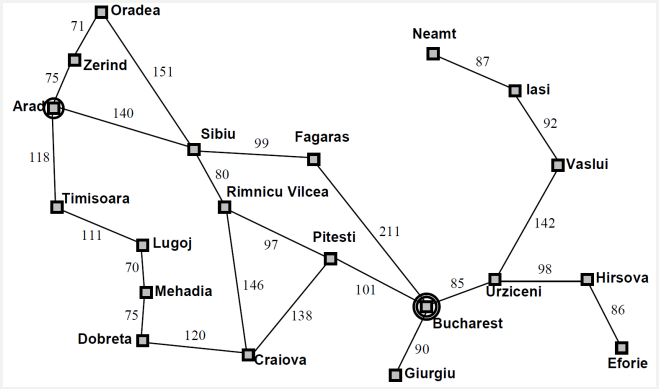


Arad

Queue: `{ node1 = < In(Arad), null, null, 0, 0 > }`

Planning trip from Arad





Expand: `node1 = < In(Arad), null, null, 0, 0 >`

Queue: {

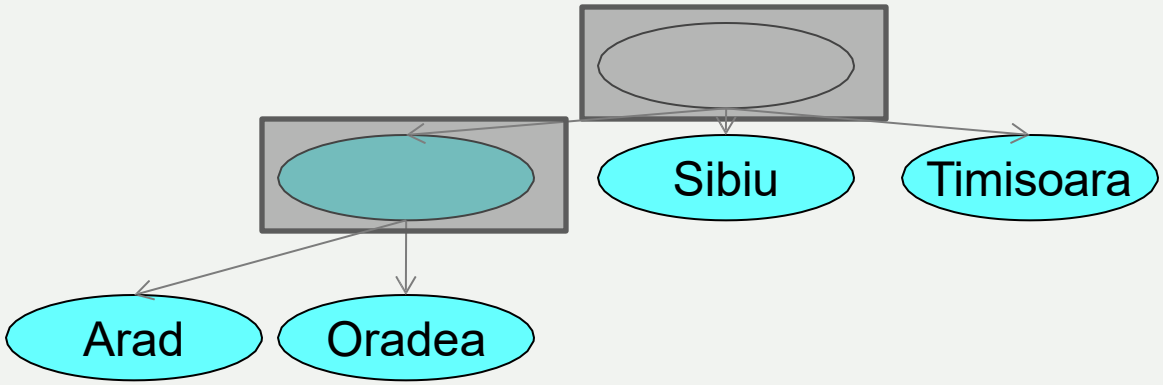
`node2 = < In(Zerind), node1, Go(Zerind), 75, 1 >,`

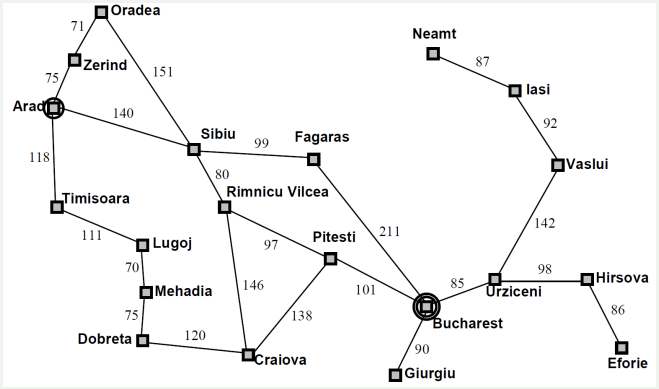
`node3 = < In(Sibiu), node1, Go(Sibiu), 140, 1 >,`

`node4 = < In(Timis…), node1, Go(Timis…), 118, 1 >`

}

Planning trip from Arad





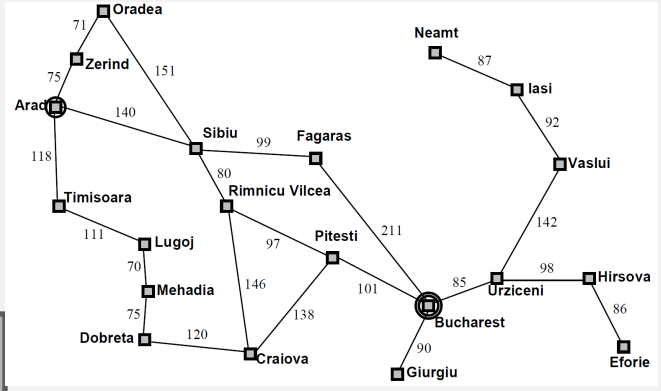
Expand: `node2 = <In(Zerind),node1,Go(Zerind),75, 1>`

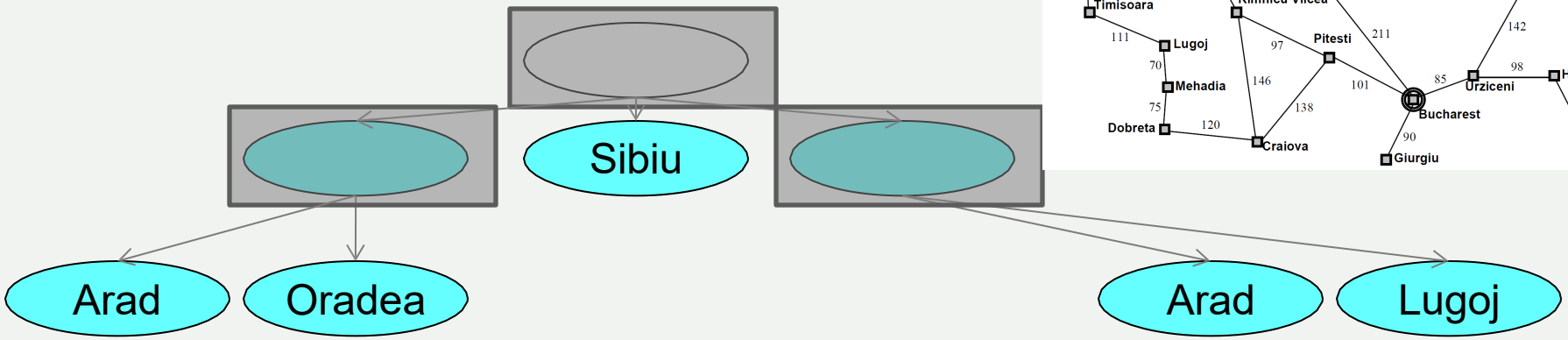
Queue: {

```
Node3 = < In(Sibiu), node1, Go(Sibiu), 140, 1 >,
Node4 = < In(Timis…), node1, Go(Timis…), 118, 1 >
Node5 = < In(Arad), node2, Go(Arad), 150, 2>
Node6 = < In(Oradea), node2, Go(Oradea),146,2>
```

}

Planning trip from Arad





Expand: `node4 = <In(Timis…),node1,Go(Timis…),118, 1>`

Queue: {

```
node3 = < In(Sibiu), node1, Go(Sibiu), 140, 1 >,
node5 = < In(Arad), node2, Go(Arad)>, 150, 2>
node6 = < In(Oradea), node2, Go(Oradea)>,146,2>
node7 = < In(Arad), node4, Go(Arad)>,236, 2>
node8 = < In(Lugoj), node4, Go(Lugoj>,229,2>
}
```

Properties of uniform-cost search

Complete? Yes if step cost > 0

Optimal? Yes

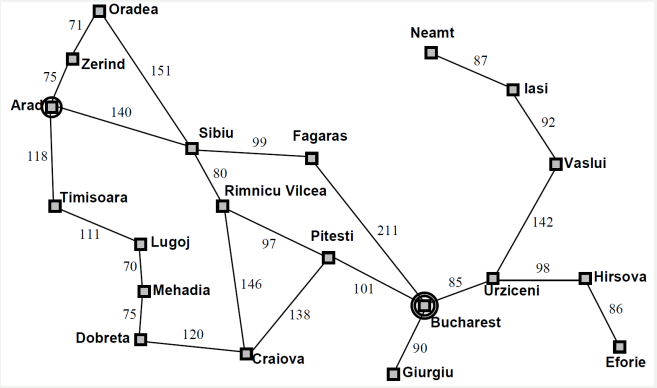
Time and space complexities

- Difficult to characterize with b and d
- Guided by path costs, not depths
- In general, time and space complexities are *worse* than for breadth-first

5/7. Depth-first search

- Expand the deepest node first
  - e.g., left-most path, 2nd left-most path, ..., right-most path
- Queue: two ways to look at it
  - LIFO Queue
  - Priority queue: highest depth first

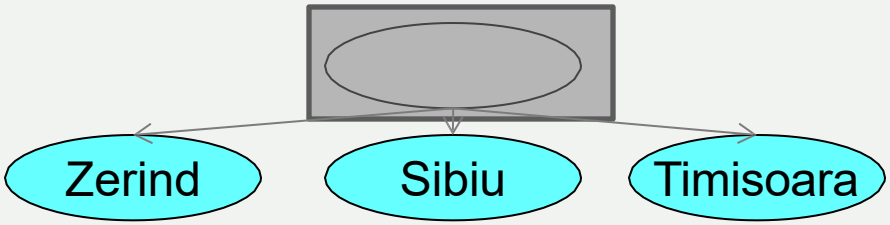
Planning trip from Arad

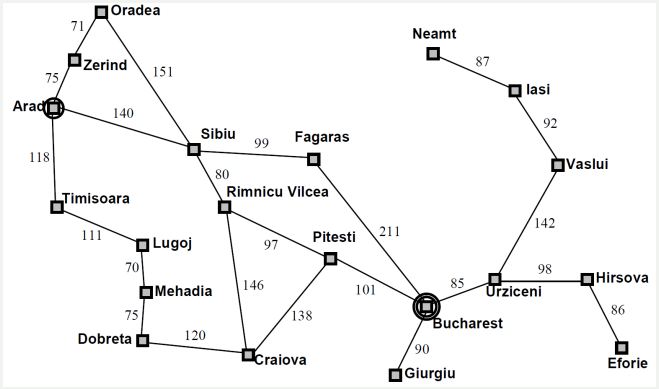


Arad

Queue: `{ node1=< In(Arad), null, null, 0, 0 > }`

Planning trip from Arad





Expand: `node1=< In(Arad), null, null, 0, 0 >`

Queue: {

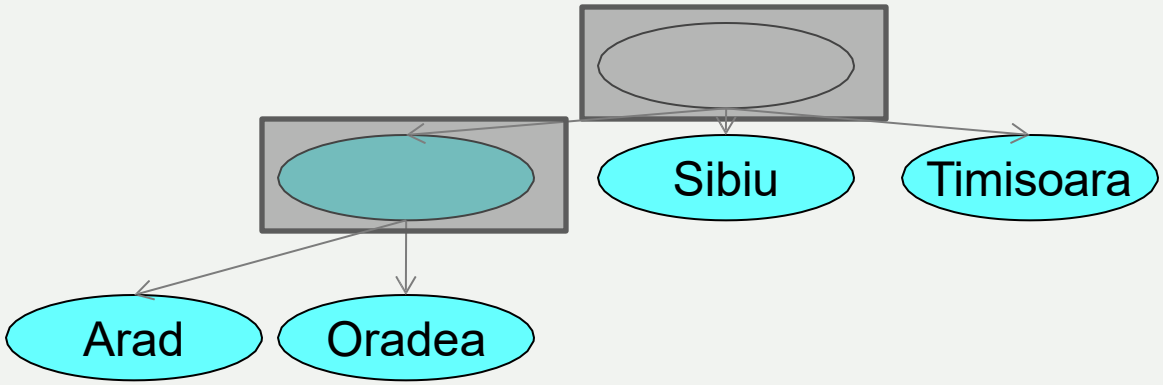
`node2=< In(Zerind), node1, Go(Zerind), 75, 1 >,`

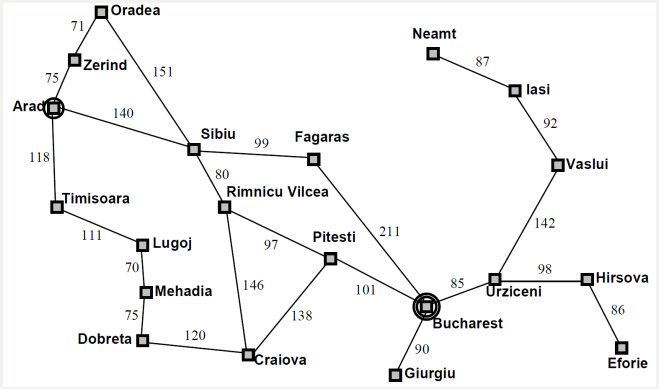
`node3=< In(Sibiu), node1, Go(Sibiu), 140, 1 >,`

`node4=< In(Timis…), node1, Go(Timis…), 118, 1 >`

}

Planning trip from Arad





Expand: `node2=< In(Zerind),node1,Go(Zerind),75, 1 >`

Queue: {

`node3=< In(Sibiu), node1, Go(Sibiu), 140, 1 >,`

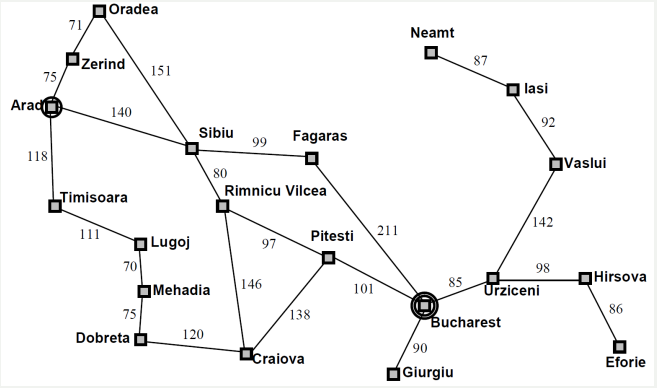
`node4=< In(Timis…), node1, Go(Timis…), 118, 1 >`

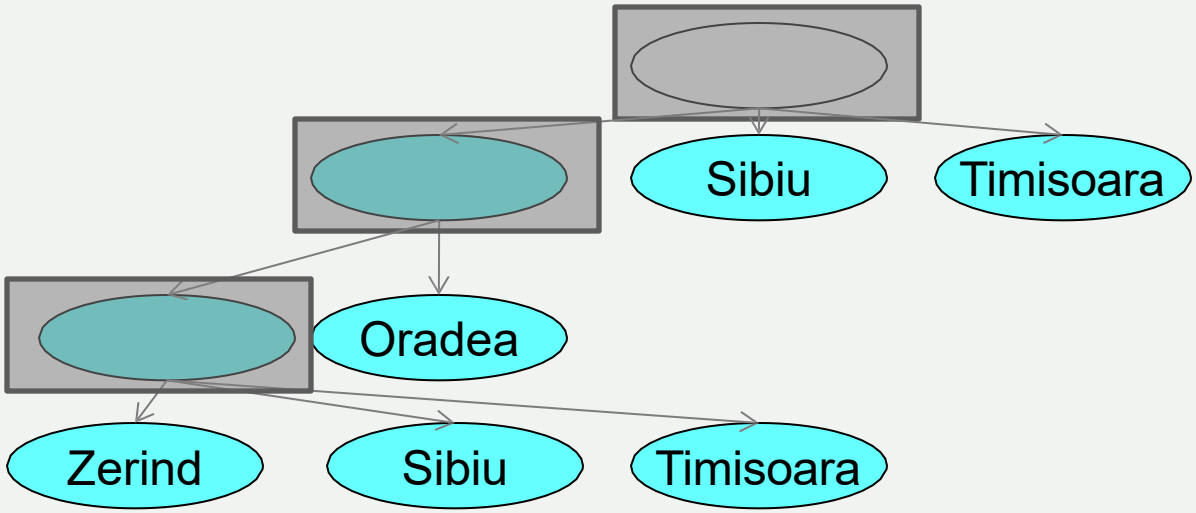
`node5=< In(Arad), node2, Go(Arad), 150, 2>`

`node6=< In(Oradea), node2, Go(Oradea),146, 2>`

}

Planning trip from Arad





Expand: `node5=< In(Arad),node1,Go(Arad),150, 1 >`

Queue: {

`node3=< In(Sibiu), node1, Go(Sibiu), 140, 1 >,`

`node4=< In(Timis…), node1, Go(Timis…), 118, 1 >,`

`node6=< In(Oradea), node2, Go(Oradea),146, 2>,`

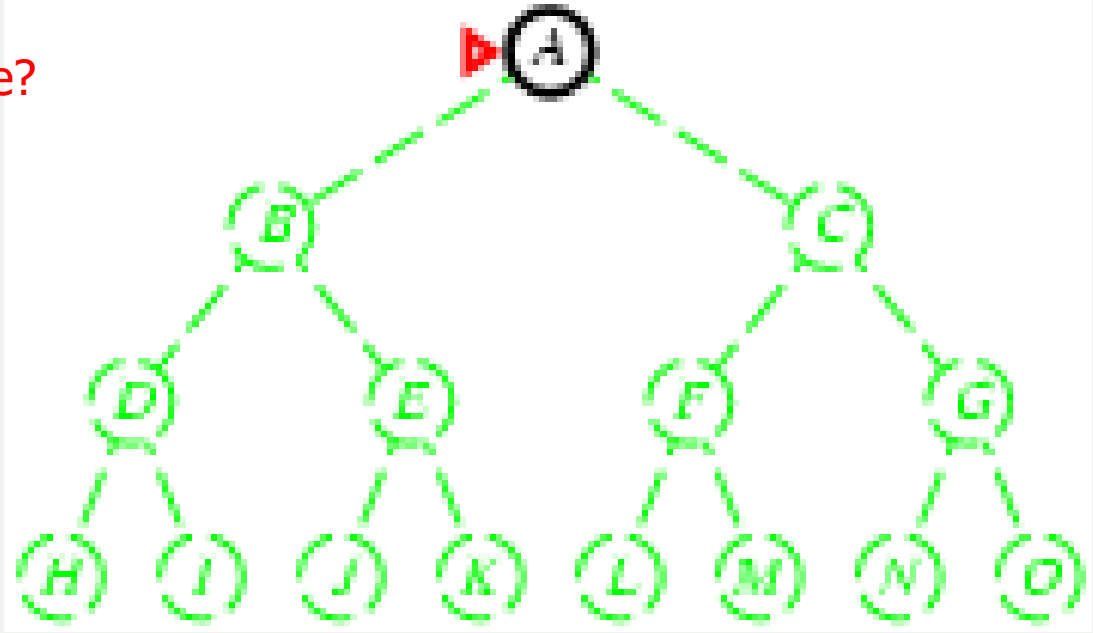
`node7=< In(Zerind), node2, Go(Zerind,225, 3>,`

… }

Another Example

- Expand *deepest* unexpanded node
- Implementation:
  - *fringe* = Last In First Out (LIFO) queue, i.e., put successors at front

Is A a goal state?

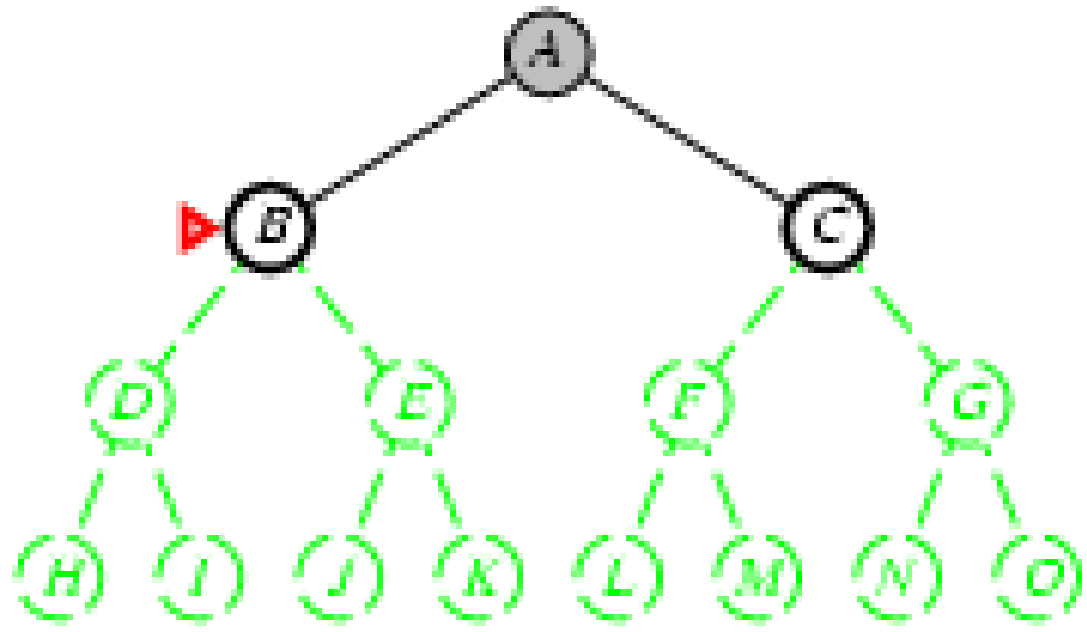


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[B,C]

Is B a goal state?

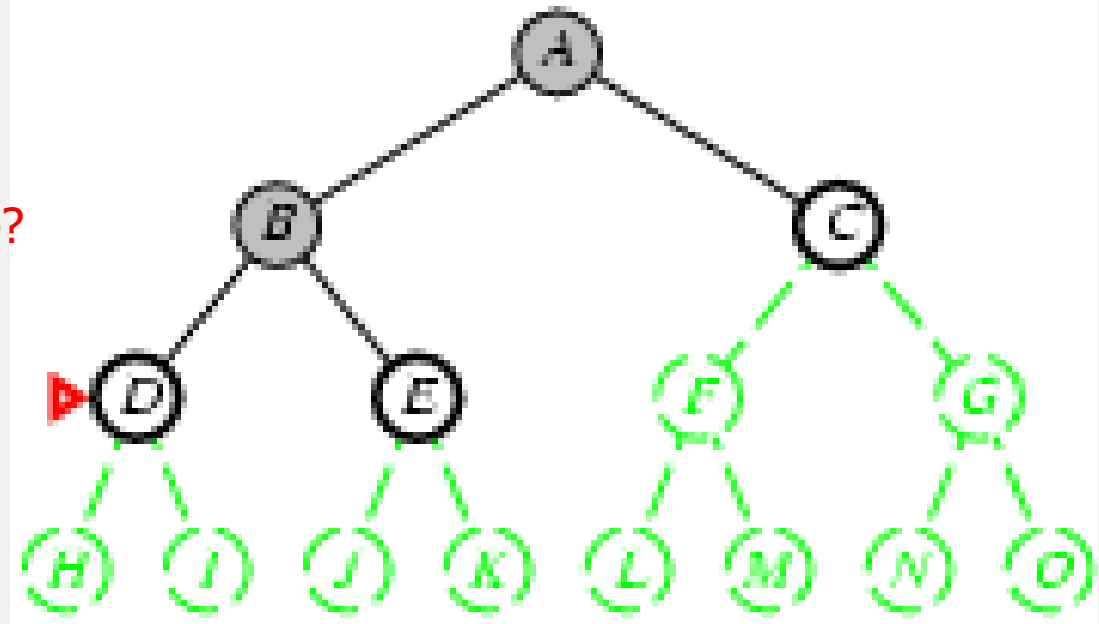


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[D,E,C]

Is D = goal state?

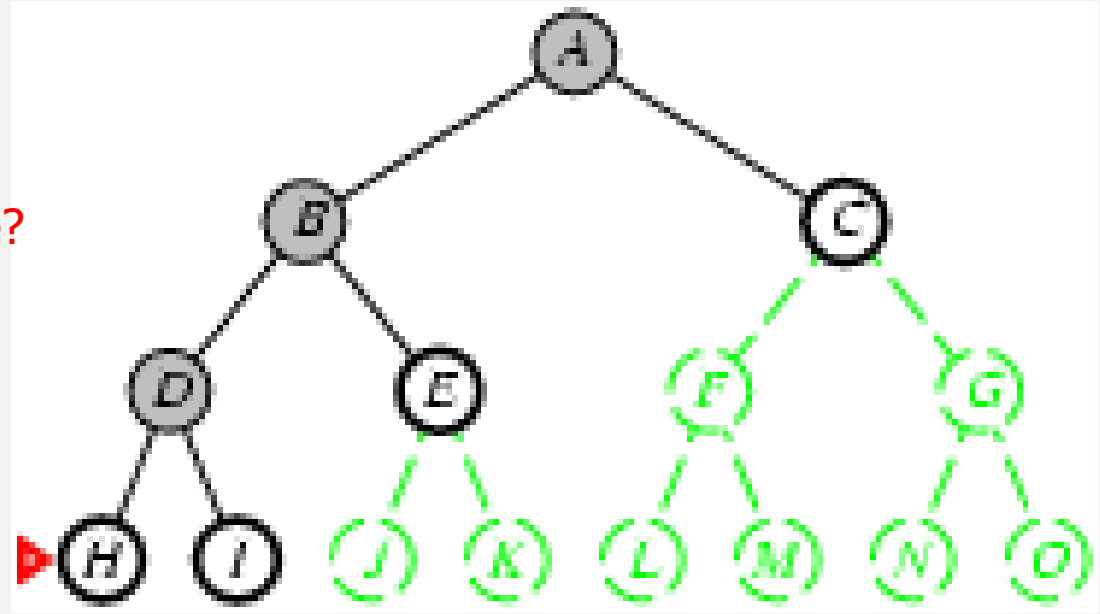


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[H,I,E,C]

Is H = goal state?

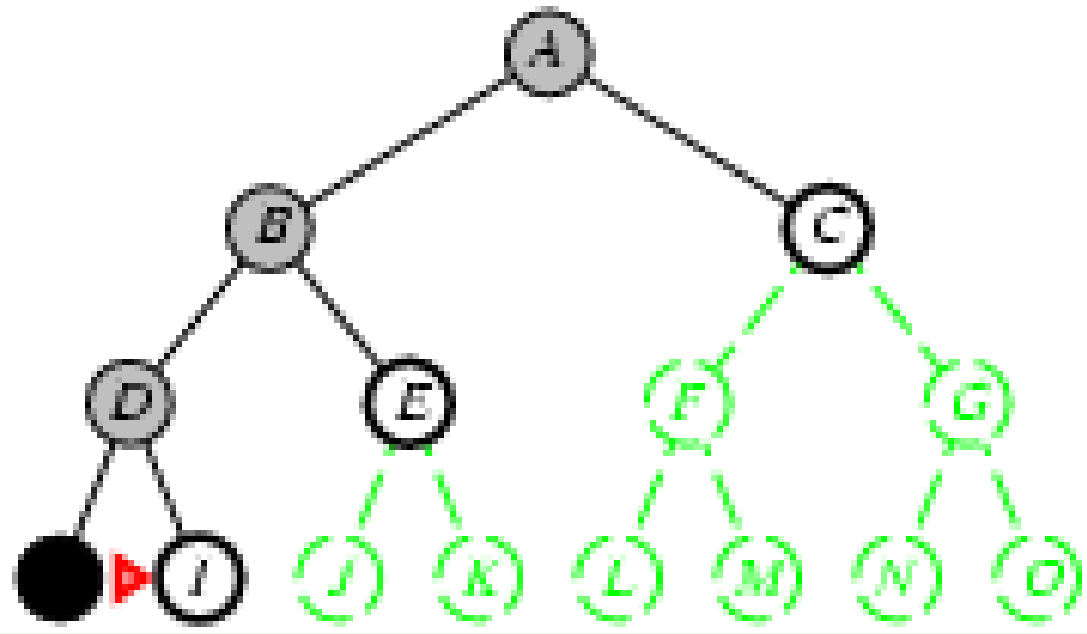


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[I,E,C]

Is I = goal state?

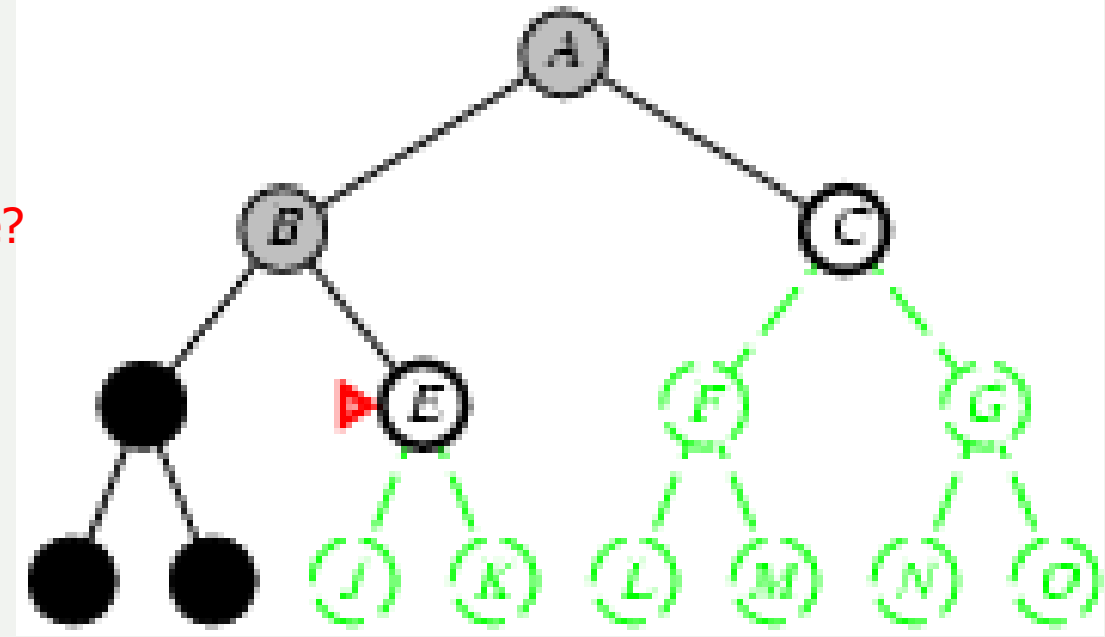


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[E,C]

Is E = goal state?

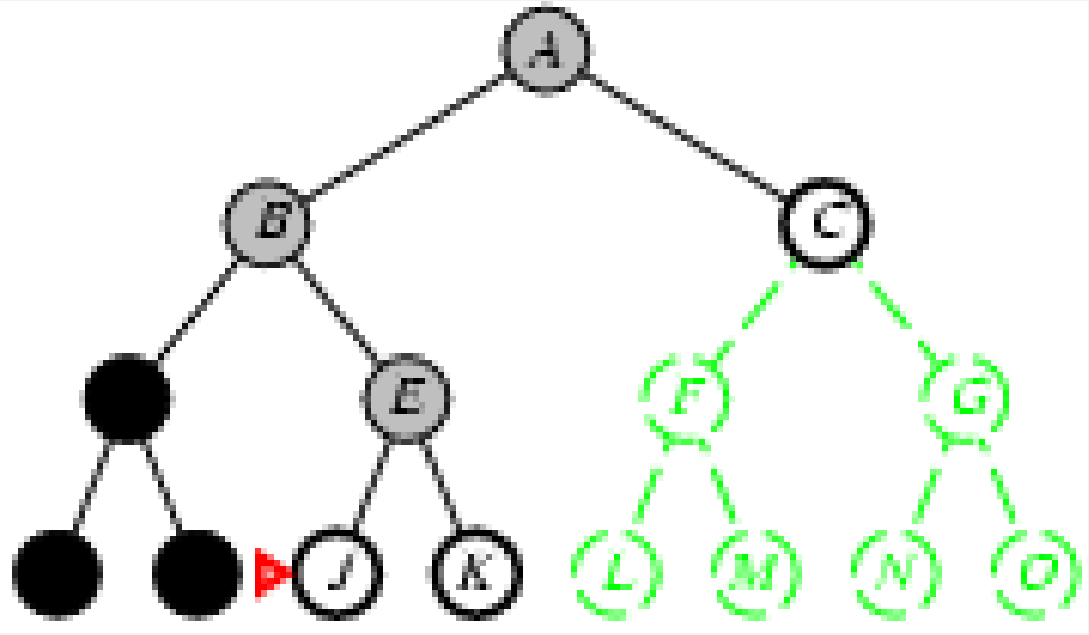


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[J,K,C]

Is J = goal state?

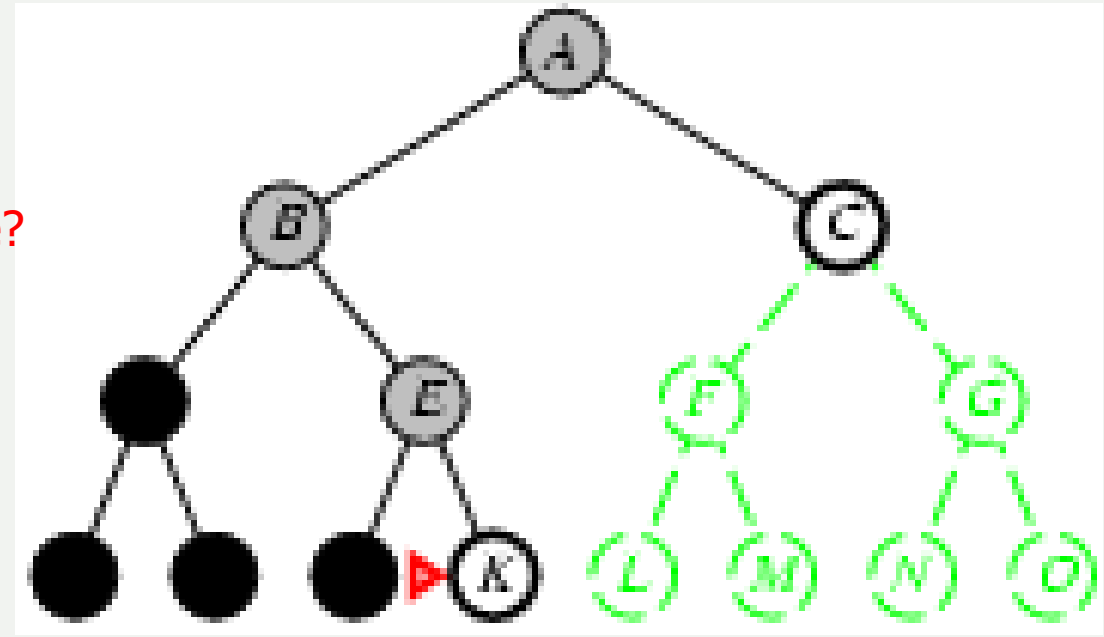


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[K,C]

Is K = goal state?

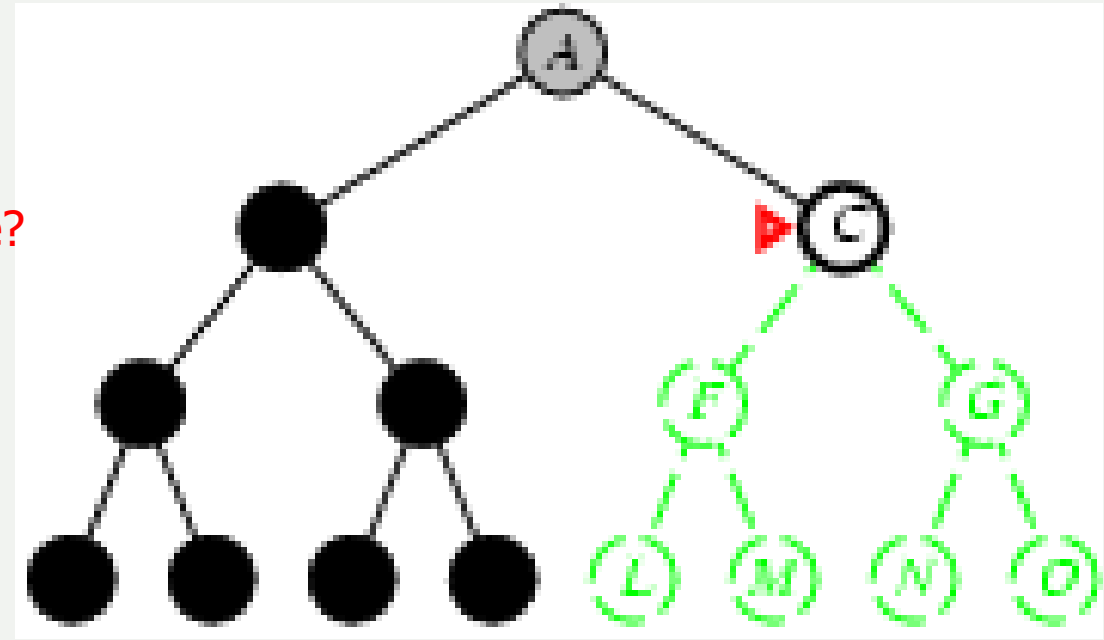


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[C]

Is C = goal state?

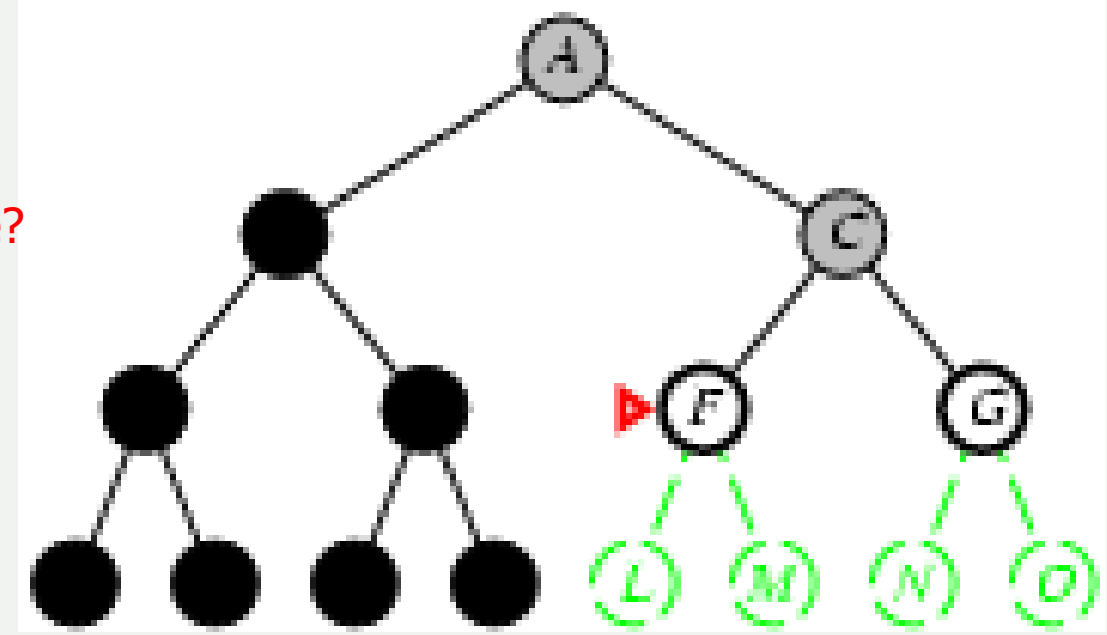


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[F,G]

Is F = goal state?

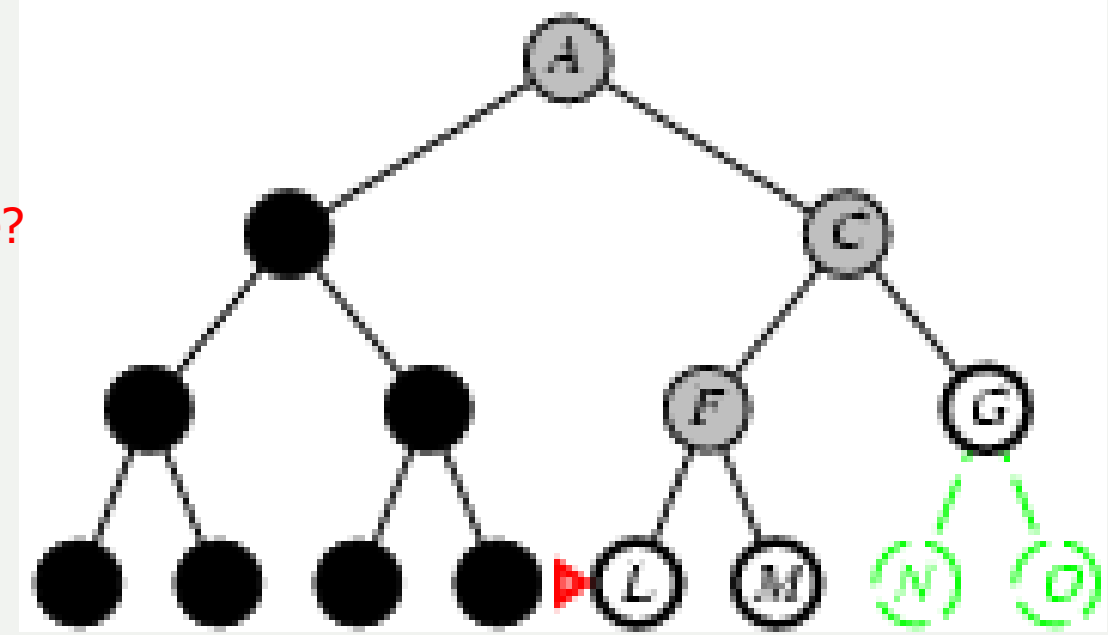


Another Example

- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[L,M,G]

Is L = goal state?

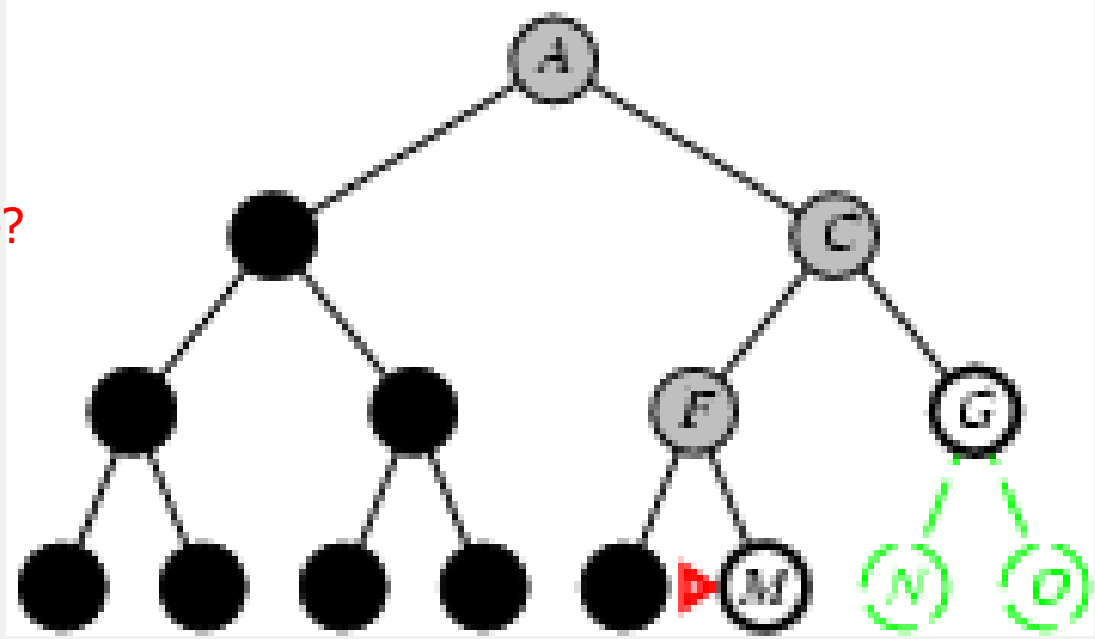


Another Example

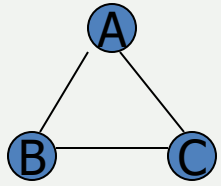
- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front

queue=[M,G]

Is M = goal state?



Complexity of depth-first search

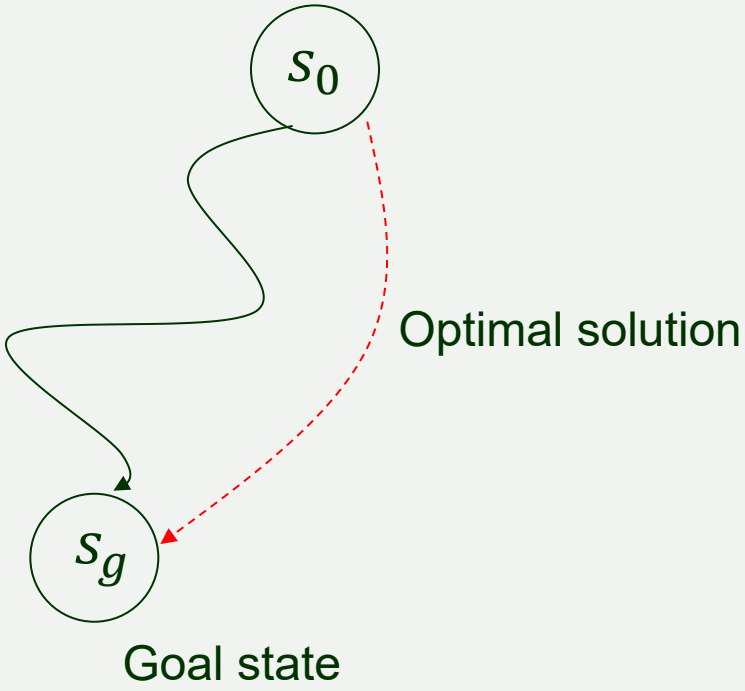


- Complete? No: fails in infinite-depth spaces

  Can modify to avoid repeated states along path
- Time? $O(b^m)$ with m=maximum depth
- terrible if *m* is much larger than *d*
  - but if solutions are dense, may be much faster than breadth-first
- Space? *O(bm),* i.e., linear space! (we only need to remember a single path + expanded unexplored nodes)
- Optimal? No (It may find a non-optimal goal first)

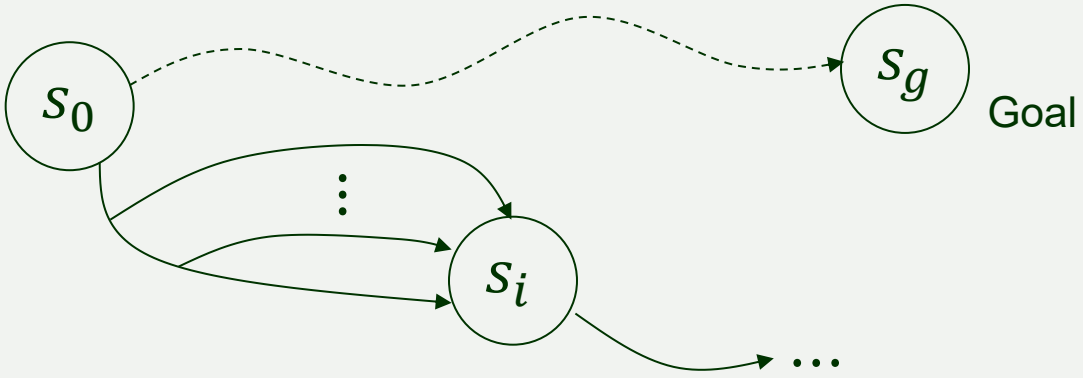
Non-Optimality of DFS

Returns the first solution it finds, even if it is not the cheapest.

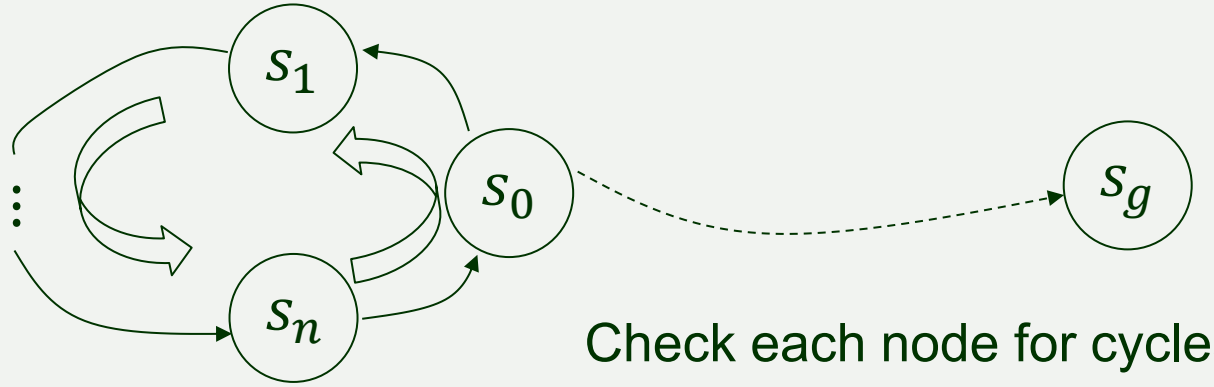


Inefficiency of DFS

- May expand the same state many times via different paths, and even systematically the entire space.



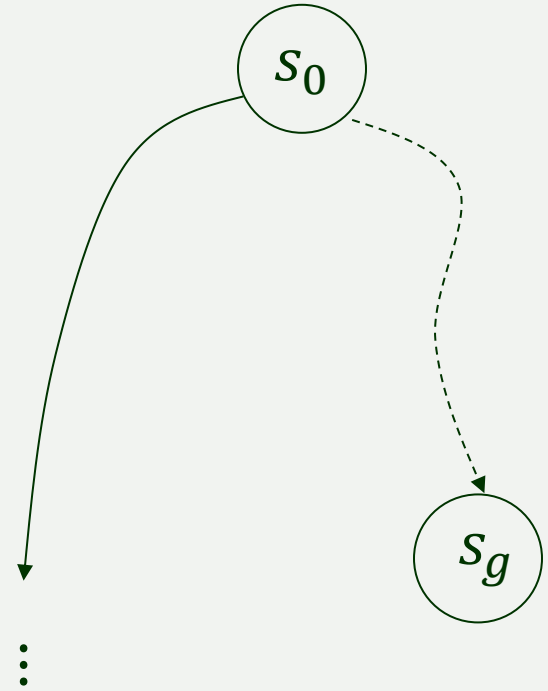
- May get stuck in an infinite loop in a cyclic state space.



Check each node for cycles.

Incompleteness of DFS

Can go down an infinite path forever.



Why Consider DFS?

- Small memory for problems.

  No need for a table of reached nodes.

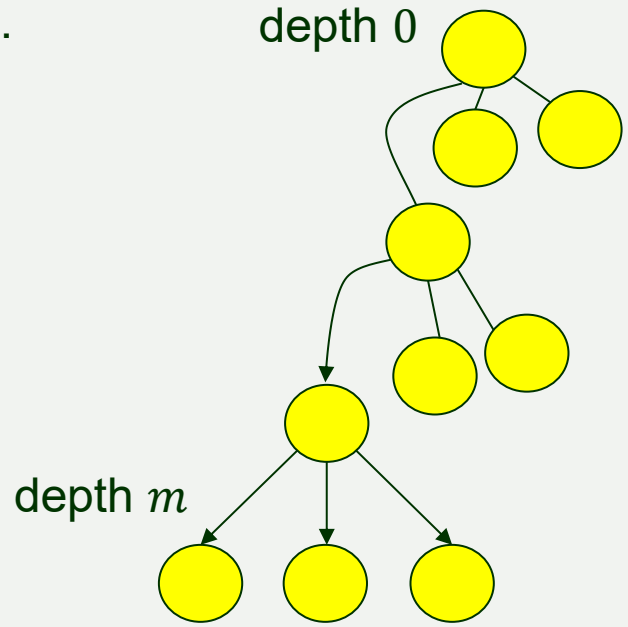
  Small frontier.

- Search time $O(\#\text{nodes})$.

- Memory consumption $O(bm)$.

  branching factor (# successors)

  maximum depth



- Workhorse of constraint satisfaction, logic programming, etc.

6/7. Modifying DFS 1 - DLS

Depth-limited search (DLS)

- Do DFS, terminate search at some search limit L
- Complete *if solution is at a depth less than L*
- Still not optimal

Depth-limited search

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Modifying DFS 2 - IDDFS

Iterative deepening depth-first search

Repeat DLS with increasing depths

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

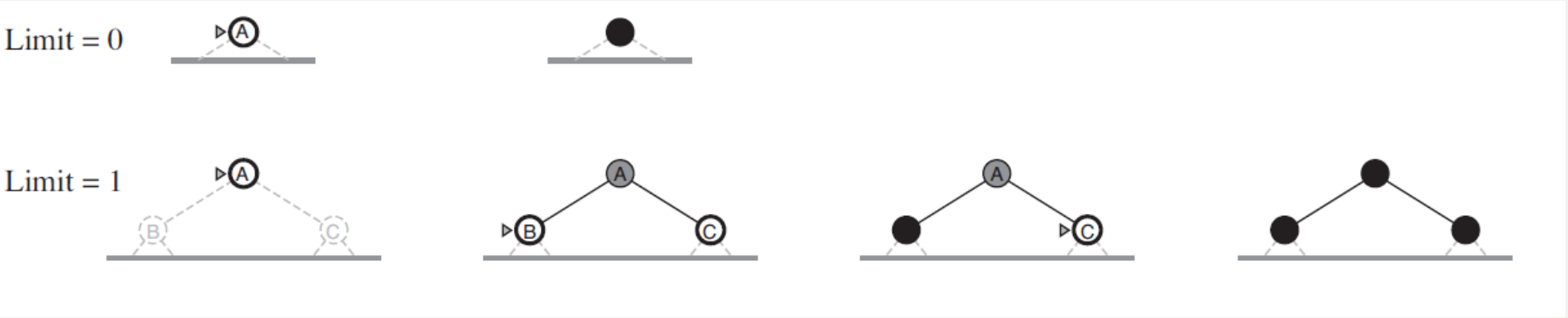
DLS vs. IDDFS

- To avoid the infinite depth problem of DFS, we can decide to only search until depth L, i.e. we don't expand beyond depth L.
  → Depth-Limited Search

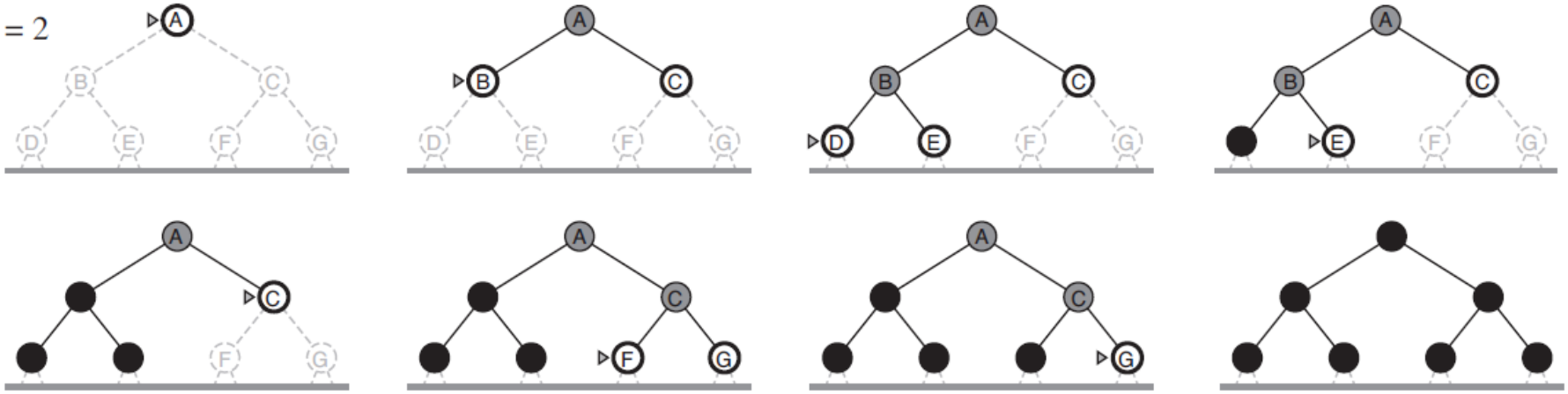
- What if solution is deeper than L? → Increase L iteratively.
  → Iterative Deepening Search

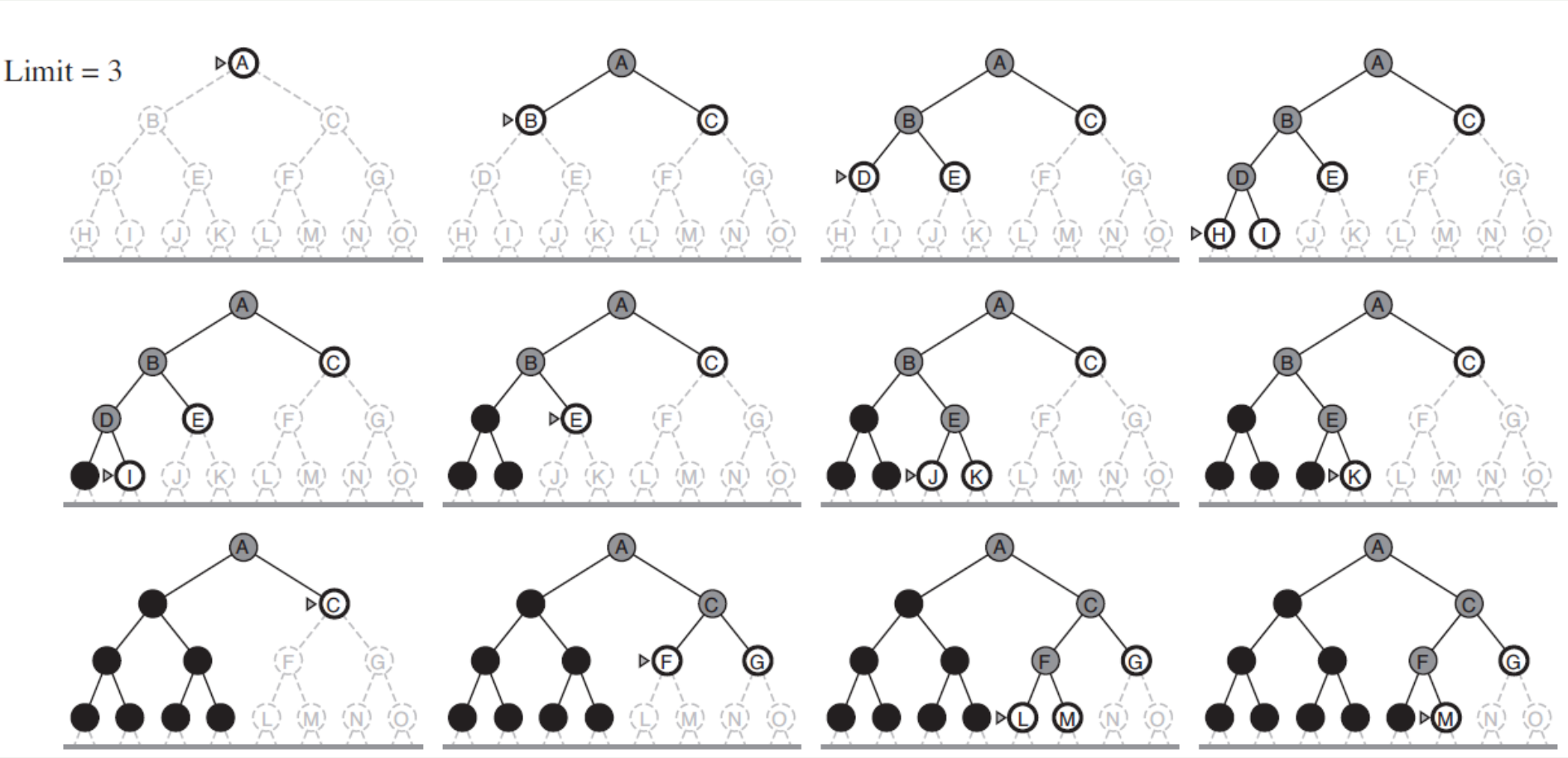
- As we shall see: this inherits the memory advantage of Depth-First search, and is better in terms of time complexity than Breadth first search.

Example



Limit = 2




Limit = 3

IDDFS

IDDFS

- Complete
- Optimal if path cost is nondecreasing w.r.t. node depth
- Space complexity: O(bd) (cf. $O(b^d)$ for BFS)
- Inherits advantages from both BFS and DFS
- It's the *preferred* search method when search space is large or d is unknown

7/7. Bidirectional search

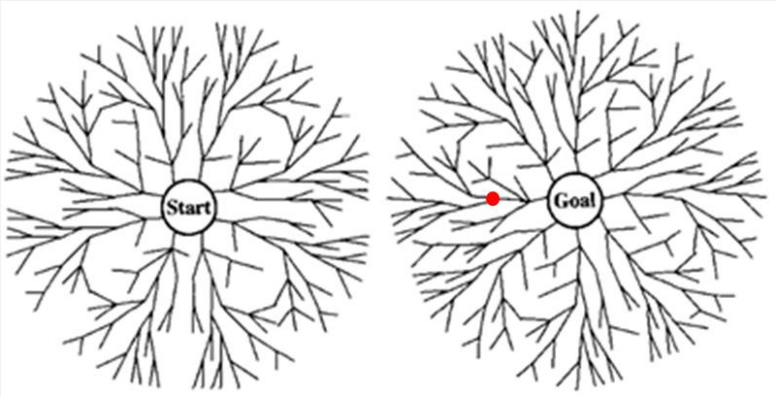
If you know the goal state (e.g., navigation)

- Search backward as well as forward
- Requires that actions are reversible
- Requires that there be one (or few) goal state(s)

Search succeeds when both searches meet at a common node

Examples

- Romania
- 8-puzzle
- Not 8-queens



- Difficult to conduct if the goal is abstractly specified.

Summary of algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]	Yes[1,4]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]	Yes[3,4]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

b: branching factor (> 1)

d: minimum depth of a solution

l: depth limit

m: maximum search tree depth

C^*: optimal solution cost

ϵ: minimum action cost

1. if b is finite, and the state space either has a solution or is finite.
2. if all costs are $\geq \varepsilon > 0$.
3. if all costs identical.
4. if both directions are breadth-first or uniform-cost.

Exercise

Romania

- Init: Sibiu, goal: Bucharest
- BFS
- UCS
- DFS
- Rules
  - Goal check is done at the time of expansion, not generation
  - Write the expanded nodes in order
  - Write the solution (=path)
  - Expand child nodes in alphabetical order if all conditions are the same